BUT RT – SAE 23

Mettre en place une solution informatique pour l'entreprise

Département Réseaux et Télécoms – IUT CV

F. ATTAL

07/02/2022

SAé 23 – Mettre en place une solution informatique pour l'entreprise

Puisqu'il est au cœur du système d'information de l'entreprise, le professionnel R&T peut être amené à développer différentes solutions informatiques : ces solutions peuvent faciliter son travail quotidien (outil pour centraliser les données d'administration de son réseau) ou être commandé pour les besoins de ses collaborateurs (annuaire des personnels, partage d'informations, ...). Ces solutions sont plus larges que le simple traitement des données (abordé au semestre 1) et visent le développement d'un outil informatique complet partant d'un cahier des charges donnés : elles incluent la gestion de données structurées (base de données, fichiers de données), leur traitement et les éléments d'interaction utilisateur via une interface conviviale et pratique. Elles peuvent être documentées grâce à des pages Web voire mises à disposition des utilisateurs directement dans leur simple navigateur Web.

Présentation de la SAE

Problématique professionnelle abordée

Le professionnel R&T peut être amené à développer des applications Web, sous forme de sites Web manipulables grâce à un navigateur Web : elles peuvent être destinées à ses collaborateurs pour mettre facilement à leur disposition un outil informatique ; elles peuvent aussi être le cœur de métier de son entreprise avec des applications destinées à des clients/commanditaires. Cette SAÉ propose à l'étudiant de s'initier au développement d'une application Web en le plaçant dans un contexte de réponse à un appel d'offres : un client cherche un prestataire pour développer une application remplissant un cahier des charges précis : par exemple, une solution pour cartographier son matériel de réseau local sous la forme d'un site web

Ressources du BUT RT mobilisées

R107 | Fondamentaux de la programmation

R108 | Bases des systèmes d'exploitation

R109 | Introduction aux technologies Web

R207 | Sources de données

R208 | Analyse et traitement de données structurée

R209 | Initiation au développement Web

R211| Expression-Culture-Communication Professionnelles 2

Description de la SAÉ

Partant des spécifications fournies par le client, vous devrez proposer, développer un prototype d'une application Web puis présenter le travail réalisé devant le jury.

Vous devez mettre en place votre environnement de travail, choisir et utiliser les technologies Web adéquates pour produire un site Web dynamique, mettre en place la gestion des données du site et programmer leur traitement.

La SAÉ pourra être réalisée par un groupe de 2 étudiants. Partant d'un cahier des charges fourni, la SAé pourra être mise en œuvre avec différentes étapes :

Phase 1 : mise en place de l'environnement de développement :

- Utilisation d'une machine Windows équipée d'un serveur Web Apache et d'une une application Web de gestion pour les systèmes de gestion de base de données MySQL

Phase 2 : réalisation documentée incluant :

- Algorithmique (script serveur, dépôt de codes)
- Technologie Web (HTML, css)
- Base de données avec manipulation de données (ajout, suppression, modification)

Phase 3 : présentation du prototype devant le jury avec rédaction d'un rapport.

Le travail demandé doit inclure à minima un livrable en anglais (résumé du rapport).

Modalités d'évaluation

L'évaluation de la SAE se basera principalement sur les Livrables suivants :

- Description du projet.
- Codes informatiques du site Web fonctionnel et dynamique
- Documentation technique, informative présentant la méthode de validation (exemple : cahier de tests, tests unitaires)
- Tutoriel d'installation et d'utilisation
- Démonstration technique commentée
- Présentation de l'outil utilisé pour le développement
- Méthode de validation
- Un rapport de 5 pages au maximum décrivant le prototype développé avec un résumé en anglais.
- Présentation du prototype devant le jury (20 min)
- Evaluation sur machine (Test de TP)

Organisation temporelle de la SAE

La SAE s'organise sur 10 semaines consécutives.

Semaine 07	**S 09**	**S 10**	**S 11**	**S 12**	**S 13**	**S 14**	**S 17**	**S 18**
CM (1h30) Présentation de la SAé								
	TP1 (1h30) Prise en main de XAMPP	TP2 (2h) Formulaires Sessions Tableaux	TP3 (4h) Interaction php / base de données	TP4 (4h) démarrage du projet				TP (2h) / Evaluation Présentation
					Projet (4h) Travail en autonomie	Projet (4h) Travail en autonomie	Projet (4h) Travail en autonomie	

Mise en œuvre

Aperçu général

La mise en œuvre de la SAE suivra les étapes suivantes :

1. Préparation de l'environnement de travail

Description du projet.
Création de l'arborescence de travail.
Vérification de fonctionnement du serveur web (XAMPP).
Mise en œuvre des tests de vérification.

2. Compréhension de la problématique

Établissement d'un schéma de fonctionnement du site avant de commencer le moindre codage.
Définition des besoins du client.
Proposition d'une solution qui répond aux besoins du client.

3. implémentation de la solution proposée

Création des bases de données.
Codage des pages html et php nécessaires.
Mise en forme des pages crées (application de style).

4. Vérification et validation

Vérification du bon fonctionnement de la solution proposée.

5. Évaluation finale

Finalisation et évaluation finale de la SAE.

TP1 : Préparation de l'environnement de travail

Cette première partie consiste en la mise en place et à la configuration des matériels et outils numériques nécessaires à la bonne réalisation de la SAE.

Préparation

Vous travaillerez sur un ordinateur de type PC sous Windows.

- Vérifiez le bon fonctionnement du serveur web (XAMPP).
  1) Lancez XAMPP Control Panel, ensuite cliquer sur Start pour lancer Apache et MySQL (Fig.1)
  2) Vérifiez que tout fonctionne bien, vous devriez avoir le message confirmation du lancement d'Apache et de MySQL (Fig. 2).

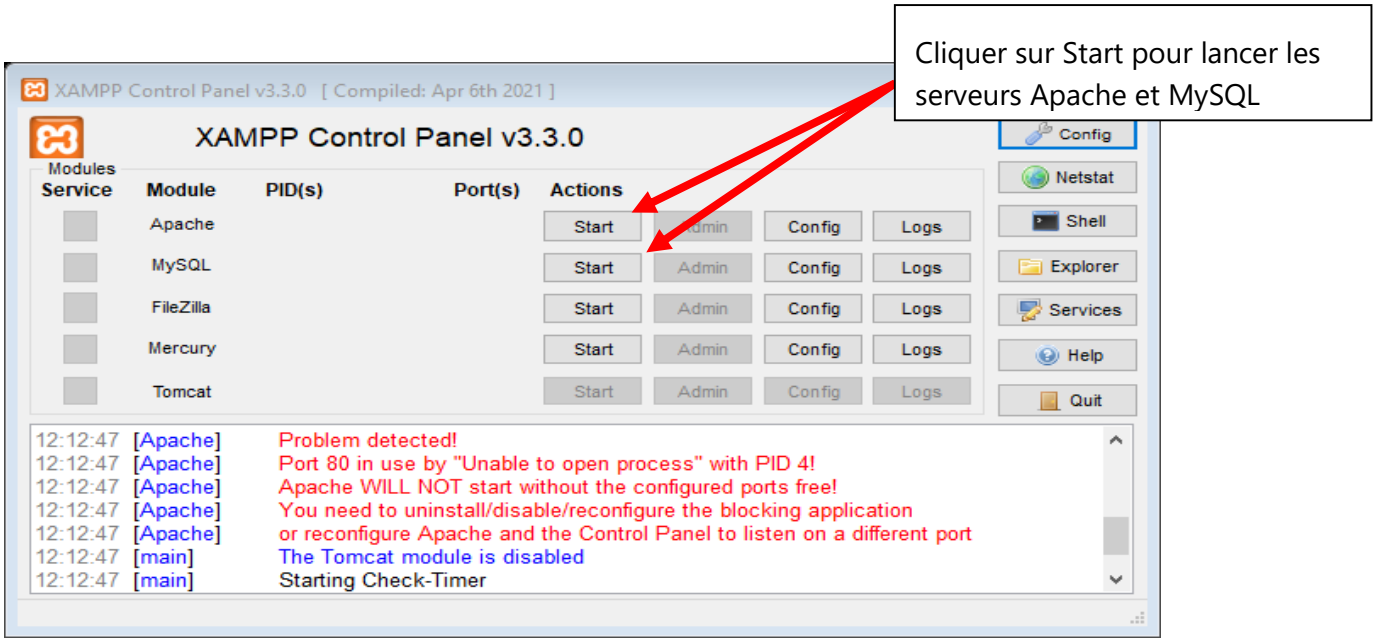


Figure 1 : XAMPP Control Panel

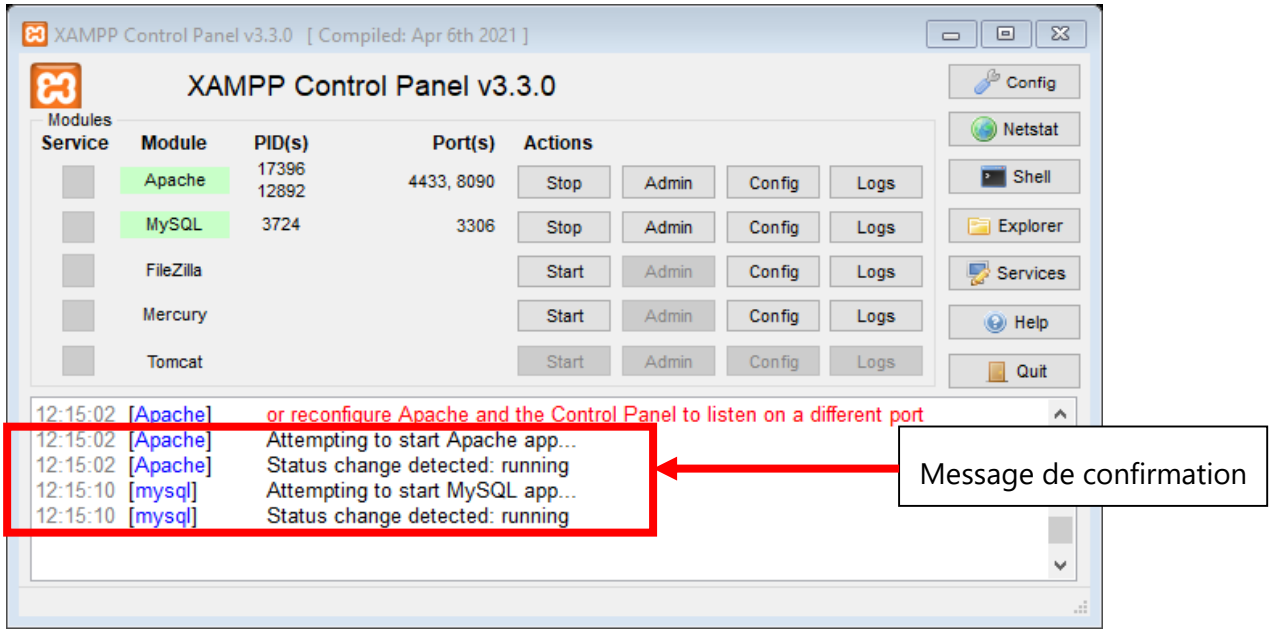


Figure 2 : Lancement des serveurs Apache et MySQL

Prise en main de XAMPP

- Créer le dossier TP1 dans C:\Xampp\htdocs
- Editez le fichier Test.html suivant dans C:\Xampp\htdocs\TP1:

```
<!DOCTYPE html>
<html> <body>
Hello !
</body> </html>
```

Lancez un navigateur et ouvrez la page 127.0.0.1:80/TP1/Test.html

1) Structure d'un fichier PHP

Toute commande PHP se termine par un point-virgule!

Tout code PHP doit être inclus dans une balise <?php ... ?>

- Editez un fichier Hello.php contenant le code ci-dessous.
- Placer la page dans l'arborescences du site web Apache (dans le répertoire Créer le dossier TP1 dans C:\Xampp\htdocs\TP1)
- Lancez un navigateur et ouvrez la page 127.0.0.1:80/TP1/Hello.php. Commentez.

```
<?php echo "Ici commence notre apprentissage de PHP !" ;
?>
```

2) Interaction avec XHTML

Il y a deux façon de procéder pour mélanger du PHP et du (X)HTML :

a) Soit insérer du PHP dans du HTML

```
<html>
  <body>
   <p> Bonjour</p>
   <?php echo "Tiens, du PHP ..."; ?>
   <p> Au revoir</p>
  </body>
 </html>
```

- Créez une page HelloPhp1.php contenant le code ci-dessus.
- Placer la page dans l'arborescences du site web Apache : dans le répertoire C:\Xampp\htdocs\TP1
- Lancez un navigateur et ouvrez la page 127.0.0.1/HelloPhp1.php . Commentez.

b) Soit insérer du HTML dans du PHP

- Créez une page HelloPhp2.php contenant le code ci-dessous.
- Placer la page dans l'arborescences du site web Apache : dans le répertoire C:\Xampp\htdocs\TP1
- Lancez un navigateur et ouvrez la page 127.0.0.1/HelloPhp2.php. Commentez.

```
<?php

echo "<html>\n";

echo "  <body>\n";

echo "    <p> Je fais du HTML avec du PHP !</p>\n";


echo "  </body>\n";

echo "</html>\n";

?>
```

3) Les variables

- Une variable commence par un dollar : $.
- Les noms de variables sont sensibles à la casse !
- $variable et $Variable sont deux variables différentes.
- Les chaînes de caractères doivent être insérées dans des guillemets.
- Exemple : Testez l'exemple suivant et vérifier que la variable $prenom affiche bien Jean

```
<?php
$prenom = "Jean" ;

echo "<html>\n";
echo "  <body>\n";
echo "<p> Je m'appelle" .$prenom. "</p>\n";
echo "  </body>\n";
echo "</html>\n";

?>
```

4) Les caractères spéciaux

Par contre, si vous insérez du HTML dans du PHP, vous devez traiter à part tous les caractères spéciaux si vous voulez que PHP ne les interprète pas. C'est par exemple le cas des guillemets dans cet exemple :

Il faut protéger les guillemets par un \ avant les guillemets

- Créez une page web TestCar.hml contenant ce code html. Testez cette page dans un navigateur

```
<!DOCTYPE html>
<html>
<body>

<table border="1" style="width:300px">
<tr>
  <td>Jill</td>
  <td>Smith</td>
  <td>50</td>
  </tr>
<tr>
  <td>Eve</td>
  <td>Jackson</td>
  <td>94</td>
</tr>
<tr>
  <td>John</td>
  <td>Doe</td>
  <td>80</td>
</tr>
</table>

</body>
</html>
```

- Créez un fichier TestCar.php pour que le tableau soit généré en php en protégeant les guillemets dans le code php :

```
<!DOCTYPE html>
<html>
<body>


<?php
echo "texte du tableau…";
?>

</body>
<html>
```

- Ouvrez le fichier TestCar.php et vérifiez que le tableau s'affiche correctement.

5) La concaténation

L'opérateur utilisé pour la concaténation de chaînes de caractères est le point.

Testez cet exemple.

```
<?php

$nom = "Pierre" ;
$prenom = "Jean" ;
echo "Mon nom est" . $prenom . " " . $nom ;

?>
```

6) Gestion des formulaires : méthodes GET/POST

On souhaite écrire un formulaire qui demande le nom et l'âge de l'utilisateur. Le bouton submit de ce formulaire provoquera l'affichage d'une page qui saluera l'utilisateur avec cette phrase : « Bonjour machin, vous avez xx ans... » (avec les bonnes valeurs, bien entendu).

- Dans un fichier TestFormulaire.html testez le code suivant permettant de réaliser le formulaire et expliquez le principe de la méthode get et le nom des variables transmises au serveur http.

```
<html><body>
        <form method="get" action=" TestFormulaire.php">
        <table>
        <tr>
          <td align="right">Votre nom</td>
          <td><input name="nom" /></td>
        </tr><tr>
          <td align="right">Votre age</td>
          <td><input name="age" /></td>
        </tr><tr>
          <td align="center" colspan="2"><input type="submit" value="Envoyer" />
        </tr>
        </table>
        </form>
        </body></html>
```

- Créez un fichier TestFormulaire.php contenant le code suivant :

```
<html>
<body>
        <?php
        echo "Bonjour ".$_GET["nom"]. " vous avez ".$_GET["age"]." ans.\n";
        ?>
</body>
</html>
```

- Expliquez le fonctionnement du script TestFormulaire.php.
- A quoi correspondent les variables $_GET["nom"] et $_GET["age"] ?
- Expliquez comment les données du formulaire sont transmises avec la méthode GET après un click sur le bouton « Envoyer ».
- Remplacer dans les fichiers TestFormulaire.html et TestFormulaire.php la méthode GET par la méthode POST (changer get en post dans l'ensemble des fichiers y compris pour les variables en respectant la casse).
- Quelle différence observez-vous lors du click sur le bouton « Envoyer » ? Expliquez.

TP2 : Formulaires, Sessions et Tableaux

Cet TP a pour objectif de vous initier aux notion de sessions, formulaire et tableaux.

Sessions et formulaires

Une session est un autre moyen de stocker des informations (dans des variables) qui peuvent être utilisées dans d'autres pages. En d'autre termes, une session permet de conserver, des informations utilisées dans une session, de façon simple et de les réutiliser dans toutes les pages d'un site.

L'utilisation du mécanisme des sessions obéit aux étapes suivantes :

- Ouverture d'une session dans chaque page ayant accès aux données à l'aide de la fonction **session_start()**.
- Définition des variables de session, c'est-à-dire des valeurs qui seront accessibles dans toutes les pages du site qui utilisent la fonction **session_start()**. Cela se réalise en utilisant le tableau super-global **$_SESSION**, dont les clés sont les noms des variables
- Lecture des variables de session dans chaque page en fonction des besoins à l'aide du tableau super-global **$_SESSION**.
- Fermeture de la session après destruction éventuelle des variables de session.

Les variables de session sont définies par le biais du tableau **$_SESSION** de la manière suivante :

$_SESSION['mavar']= mavaleur;

Ou encore :

$_SESSION['mavar']= $mavariable;

La valeur est ensuite lisible dans toutes les pages en écrivant simplement :

echo $_SESSION['mavar'];

Exercice 1

- Créez un fichier page1.php contenant le code suivant :

```
<?php
session_start();
$nom="Jean";
$_SESSION['nom']=$nom;
echo "<a href=\"page2.php\">Vers la page 2</a>";
?>
```

- Créez un fichier page2.php contenant le code suivant :

```
<?php
session_start();
echo "<br /> Bonjour ",$_SESSION['nom'];
?>
```

- Expliquez le fonctionnement du script page1.php.
- Expliquez le résultat obtenu, après un clic sur *Vers la page 2*.

Exercice 2

- Complétez le fichier Test_session.php qui permet de générer un formulaire de saisie des deux codes couleur, préférés d'un visiteur du site, pour la couleur de fond et le texte. Enregistrez-les dans des variables de session pour afficher la page Test_couleurs.php avec ses couleurs préférées.

Test_session.php

```
<!DOCTYPE html>
 <head>
  <meta charset="UTF-8" />
  <title>Couleurs du site avec des sessions</title>
 </head>
 <body>
  <div>
   <form ……">
```

À compléter pour avoir le rendu illustrer sur la figure 1.

1) ***Deux zones de saisies***
2) ***Deux bouton (un pour transmettre les saisies et l'autre pour les effacer)***

```
  </form>

  <p>Contenu de la page principale <br />
   <a href= À compléter > Lien vers la page Test_couleurs  qui aura ces
couleurs</a>
  </p>
  </div>
 </body>
</html>

<?php
if(isset($_POST['fond']) and isset($_POST['texte']))
{
 À compléter (enregistrement des couleurs préférés du visiteur dans des
variables session)

}
?>
```

Figure 1 : Formulaire de saisie des couleurs

.

Test_couleurs.php

```
<?php
À compléter (récupération des variables sessions)
?>
<!DOCTYPE html>
<html>
 <head>
```

```
  <title>Couleurs du site avec des sessions</title>
  <style type="text/css" >
  Body {background-color: <?php À compléter ?> ; color: <?php echo À completer ?> ;}
  </style>
 </head>
 <body>
  <p>Contenu de la page B avec les couleurs choisies <br />
   <a href="Test_session.php">Retour vers la page principale</a>
  </p>
 </body>
</html>
```

Exercice 3

Créez trois fichiers, ***pageaccueil.html***, ***pageverification.php*** et ***pagecommande.php***.

Sur la page d'accueil, ***pageaccueil.html***, le script crée un formulaire classique d'authentification, dans lequel le visiteur doit saisir son login et son mot de passe.

Le script ***pageverification.php*** , vérifie si le login et le code sont corrects, auquel cas il donne l'accès à la page de commande, sinon il informe l'utilisateur que le login ou mot de passe est incorrecte (utilisé une alerte JavaScript).

Le formulaire de saisie comprend trois zones de saisie ainsi que trois boutons Submit, chacun correspondant à une action particulière.

- Le bouton ***Ajouter*** ajoute un article à la commande.
- Le bouton ***Voir Mon panier*** affiche l'ensemble de la commande et le montant total.
- Le bouton ***Supprimer*** supprime le contenu de mon panier.

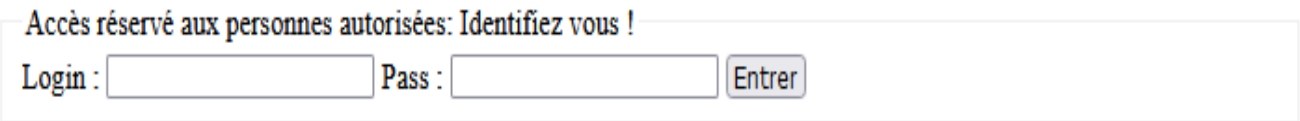
pageaccueil.html

```
<!DOCTYPE html>
<html >
<head>
<title>SESSIONS login</title>
</head>
<body>
<div>
<form method="post" A completer >
```

À compléter

```
</form>
</body>
</html>
```

Résultat attendu



pageverification.php

```
<?php

error_reporting(0);

if($_POST['login']=="login" && $_POST['pass']=="motdepasse")

{
 echo "<script  type=\"text/javascript\"> À compléter   </script>"; // pour lancer la page pagecommande.php

} else

{
        echo "<script> alert(\"À compléter\") </script>"; // pour informer l'utilisateur que le login ou le mot de passe est incorrecte
           echo "<script> À compléter</script>"; // pour revenir à la page d'accueil


}
```

```
?>
```

pagecommande.php

```
<!DOCTYPE html >
<html >
<head>
<title>Géstion de panier</title>
</head>
<body>
< !--création du formulaire : Le résultat attendu est illustré sur la figure 1 -->
<form action="<?php $_SERVER['PHP_SELF'] ?>" method="post" >
<fieldset>
<legend><b>Saisies d’articles</b></legend>
<table>
<tbody>
<tr>
<th>code : </th>
<td> <input À compléter /></td>  < !-- zone de saisie du code de l’article -->
</tr>
<tr>
<th>article : </th>
<td><input À compléter /></td> < !-- zone de saisie du nom de l’article -->
</tr>
<tr>
<th>prix :</th>
<td><input À compléter /></td> < !-- zone de saisie du prix de l’article -->
</tr>
<tr>
<td colspan="3">
```

<input **À compléter** /> < !-- Bouton Ajouter pour ajouter un article à la commande. -->

<input **À compléter** /> < !-- Bouton Voir Mon panier pour afficher l'ensemble de la commande et le montant total. -->

<input **À compléter** /> < !-- Bouton Supprimer pour supprimer le contenu de mon panier. -->

```
</td>
</tr>
</tbody>
</table>
</fieldset>
</form>
</body>
</html>

<?php
session_start();
error_reporting(0);
//AJOUTER
if($_POST["envoi"]=="Ajouter" && $_POST["code"]!="" &&
$_POST["article"]!="" && $_POST["prix"]!="")
{
// À compléter : récupération des saisies du visiteur

$_SESSION['code']= $_SESSION['code']."//".$code;
$_SESSION['article']= $_SESSION['article']."//".$article;
$_SESSION['prix']= $_SESSION['prix']."//".$prix;
}
// Voir Mon panier
if($_POST["envoi"]=="Voir Mon panier")
```

```
{
echo "<table border=\"1\" >";
echo "<tr><td colspan=\"3\"><b>Récapitulatif de votre commande</b></td>";
echo "<tr><th> code </th><th> article </
th><th> 
prix </th>";
$total=0;
$tab_code=explode("//",$_SESSION['code']);
$tab_article=explode("//",$_SESSION['article']);
$tab_prix=explode("//",$_SESSION['prix']);
// affichage du panier. Le résultat attendu est illustré sur la figure 2
for($i=1;$i<count($tab_code);$i++)
{
echo "<tr> <td> À compléter  </td> <td> À compléter  </td><td>
".sprintf("%01.2f", $tab_prix[$i])."</td>"; // À compléter pour constituer le
panier.
$prixtotal += ;  // À compléter (calculer le prix total)
}
echo "<tr> <td colspan=2> PRIX TOTAL </td> <td>".sprintf("%01.2f",
$prixtotal)."
</td>";
echo "</table>";
}
//Supprimer
if($_POST["envoi"]=="Supprimer")
{
session_unset();
session_destroy();
echo "<h3>La session est terminée</h3>";
}
```

```
$_POST["envoi"]="";
?>
```

Saisies d'articles

code :

article :

prix :

Ajouter | Voir Mon panier | Supprimer

Figure 1. Formulaire de saisie

Récapitulatif de votre commande		
code	**article**	**prix**
1	Iphone 13	1000.00
2	PC Portable	1400.00
PRIX TOTAL		2400.00

Figure 2. Affichage du panier

TP3 : Formulaires, Sessions et Tableaux

Cet TP a pour objectif de vous initier aux notion de sessions, formulaire et tableaux.

Sessions et formulaires

Pour que la base de données MySQL soit accessible à partir des pages d'un site, il faut pouvoir l'utiliser par l'intermédiaire d'un script. MySQL est utilisable par d'autres langages que PHP, Java par exemple. Le couple PHP-MySQL est cependant le plus répandu sur le Web.

L'accès à une base MySQL et son utilisation, qu'il s'agisse d'insérer, de modifier ou de lire des données, suit les étapes ci-dessous :

1. Connexion au serveur MySQL.

2. Envoi de diverses requêtes SQL au serveur (insertion, lecture, suppression ou mise à jour des données).

3. Récupération du résultat d'une requête.

4. Fermeture de la connexion au serveur

Exercice 1 :

Soit le Modèle Logique des Données d'une base de données d'un magasin :

article(id_article, designation, prix, catégorie)

1) La colonne id_article: type CHAR(10), NOT NULL et PRIMARY KEY.
2) La colonne designation : type VARCHAR(100), NOT NULL.
3) La colonne prix : type DECIMAL(8,2), NOT NULL.
4) La colonne catégorie : type ENUM. Elle comporte une liste de six valeurs possibles ('Tous', 'Vidéo', 'Photo', 'Téléphonie', 'Informatique', 'Divers'), NOT NULL et la valeur par défaut 'tous'.

- Donnez le script permettant d'établir la connexion au serveur Mysql et la création d'une base de données ***''mydb''***, avec

  $servername = "localhost";

  $username = "root";

  $password = "";

- Donnez le script ***Creat_Client_Table.php*** qui permet la création de la table ***article*** dans la base de données ***''mydb''*** ainsi que l'insertion des données suivants :

id_article	designation	prix	catégorie
HXR-NX200	Caméscope Sony	2419 €	Vidéo
Alpha_7_iii	Appareil Photo Sony	2300 €	Photo
IPhone_13	IPhone 13 Pro Max Appel	1609 €	Téléphonie
M15_R4	Pc Portable Dell	3147 €	Informatique
WH1000XM4	Casque Bluetooth Sonny	380 €	Divers

- Lancez 127.0.0.1:80/phpmyadmin et vérifiez dans phpMydamin la création de la table ***article***.

Exercice 2

L'objectif de cet exercice est de développer une interface web (comme illustré sur la figure 1) qui permet la gestion de la base de données ***article***. Cette application est destinée à un vendeur pour lui donner la possibilité de modifier, ajouter, ou supprimer un article.

Articles Disponibles

Code Article	Désignation	Prix unitaire		
Alpha 7 iii	Appareil Photo Sony	2300.00 €	Modifier	Supprimer
HXR-NX200	Caméscope Sony	2419.00 €	Modifier	Supprimer
IPhone_13	IPhone 13 Pro Max Appel	1609.00 €	Modifier	Supprimer
M15_R4	Pc Portable Dell	3147.00 €	Modifier	Supprimer
WH1000XM4	Casque Bluetooth Sonny	380.00 €	Modifier	Supprimer

Ajouter un Article

Figure1 : interface web pour la gestion de la base de données article

Ci-dessous le fichier source de la page ***page_accueil_vend.***php, à compléter, permettant d'afficher le détail sur les articles disponibles :

```
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>
<title> Gestion de la base article </title>
</head>
<body>
<h1>  Articles Disponibles </h1>
<br />
<table border="1"  >
<tr>
<th>Code Article</th>
<th>Désignation</th>
<th>Prix unitaire</th>
<th> </th>

<?php

$servername = "localhost";
$username = "root";
$password = "";
$dbname = "mydb";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}

//Création de la requète
$requete = ; // À compléter : requête SQL qui permet de sélection l'
id_article, la designation et le prix à partir de la table article
$result=$conn->query($requete);
$nbart=$result->num_rows;

while($tab = $result->fetch_assoc())
{
echo "<tr> <td>", À compléter,"</td> <td>", À compléter,"</td>
```

<td align=\"right\">", **À compléter**," € </td>"; ***//* pour insérer dans le tableau html les valeurs récupérées à partir de** $tab

```
$modif="modifier.php?id_article=".$tab['id_article']."&designation=".$tab['design
ation'];
$supp= "supprimer.php?id_article=".$tab['id_article'];
echo "<td> <a href=$modif>Modifier</a> ";
echo " ";
echo " ";
echo "<a href=$supp >Supprimer</a> </td>";
echo "</tr>";

  }
 echo "</table>";

 $conn->close();
  ?>

  <h2>  <a href="ajouter.php">Ajouter un Article</a> </h2>
  </body>
</html>
```

On remarque en particulier les lignes :

```
echo "<td> <a href=$modif>Modifier</a> ";
echo "<a href=$supp >Supprimer</a> </td>";
```

avec

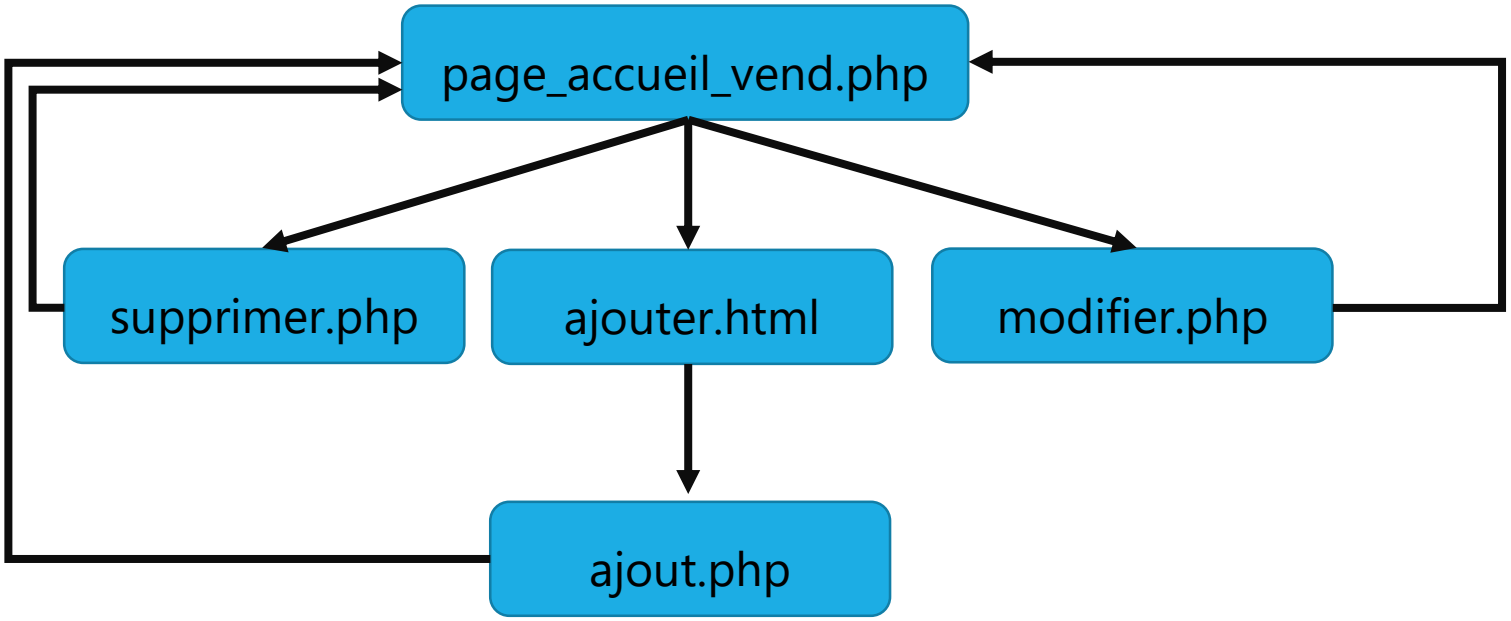
```
$modif="modifier.php?id_article=".$tab['id_article']."&designation=".$tab['design
ation'];
$supp= "supprimer.php?id_article=".$tab['id_article'];
```

A quoi sert ?id_article=".$tab['id_article']."&designation=".$tab['designation']; ?

Que se passe-t-il lorsque l'on fait un click sur le bouton modifier ou supprimer ?

Comment la variable id_article est-elle récupérée dans les pages supprimer.php et modifier.php ?

On souhaite développer les pages associées à la modification (modifier.php), la suppression (suppression.php), l'ajout d'un nouvel article (ajouter.html et ajout.php). Ci-dessous les liens entre les pages :



On souhaite maintenant réaliser la page modifer.php

Le code de cette page, à compléter, est le suivant :

```
<?php
if (isset($_POST['enreg']) ) {

$servername = "localhost";
$username = "root";
$password = "";
$dbname = "mydb";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}

$requete= ;//À compléter :  requête SQL qui permet de mettre à jour la
base article par les informations saisies par le vendeur.

$conn->query($requete);
 $conn->close();
header('Location: page_accueil_vend.php');
}
?>

<html>
<head>
<title> Modification d'un article </title>
</head>
<body>
<h1> Modifier un article</h1>
<form action="modifier.php" method="post">
<table>
<tr> <td align="right">Designation</td>
```

```
<td><input name="designation" value= "<?php echo $_GET['designation']; ?>"
/></td>
</tr>

<tr> <td align="right">Prix</td>
<td><input name="prix" value= " " /></td>
</tr>


<tr> <td><input type="hidden" name="id_article" value= “À completer pour
transmettre la valeur de  id_article ” /> </td>
</tr>

<td align="center" colspan="2"><input type="submit" name="enreg"
value="Modifier"/>
</tr>
</table>
</form>
</body>
</html>
```

Pourquoi le type hidden ?
Expliquez le fonctionnement du script modifier.php

On souhaite maintenant réaliser la page ajouter.html

Complétez la page ajouter.htlm pour avoir la résultat illustré sur la figure .

```
<!DOCTYPE html>
<html>
<head>
<title>Ajout d'un article</title>
</head>
<body>
<h2>Ajouter un article</h2>
<form action="ajout.php" method="post">
```

```
<fieldset>
<legend><b>Insérer le nouveau article </b></legend>
<table>
<tbody>
<tr>
<td>Id_article  :
</td>
<td> <input À compléter /><br /> < !-- zone de saisie de l' Id_article  de l'article
-->

</td>
</tr>
<tr>
<td>Designation : </td>
<td> <input À compléter /><br /> < !-- zone de saisie de désignation  de
l'article -->

 </td>
</tr>

<tr>
<td>Prix : </td>
<td> <input  À compléter /><br /> < !-- zone de saisie du prix  de l'article -->

 </td>
</tr>

<tr>
<td>Catégorie : </td>
 <td><select name="categorie" size="1">
 <option value="video"> Vidéo</option>
 <option value="photo"> Photo</option>
 <option value="informatique"> Informatique</option>
  <option value="Téléphonie"> Téléphonie</option>
 <option value="divers"> Divers</option>
 </select></td>

</tr>

<tr>
<td colspan="2">
<input type="submit" value="Enregistrer" name="envoi" />
```

```
<input type="submit" value="Page d'accueil" name="envoi" />
 </td>
</tr>
 </tbody>
</table>
</fieldset>
</form>

</body>
</html>
```

Figure : formulaire d'insertion d'un nouvel article

ajout.php

```
<?php

// À compléter : Connexion au serveur MySQL



if($_POST['envoi']=="Enregistrer"
&&$_POST["id_article"]&&$_POST["designation"]&&$_POST["prix"]&&$_POST
["categorie"])

{
 $id_article= $_POST["id_article"];
 $designation= $_POST["designation"];
 $prix= $_POST["prix"];
 $categorie=$_POST["categorie"];

$requete =// " À compléter :  requête SQL qui permet d'insérer l' id_article,
la designation, le prix et la categorie dans la table article ";


 $conn->query($requete);

header('Location: page_accueil_vend.php');
  }
else if($_POST['envoi']!=="Page d'accueil"){
        echo "<script  type=\"text/javascript\">alert('<?php Erreur de saisie ?>');
</script>";
        echo "<script  type=\"text/javascript\"> window.location='ajouter.html'
</script>";
}
else
{
       echo "<script  type=\"text/javascript\">
window.location='page_accueil_vend.php' </script>";
}

$conn->close();
?>
```

On souhaite maintenant réaliser la page supprimer.php
Complétez le code de la page supprimer.php pour supprimer un enregistrement de la table article à partir de son id.

```
<!DOCTYPE html>
<html>
<head>
<title>Suppression d’un article</title>
</head>
<body>
<h1>Suppression d’un article</h1>
<?php

// À compléter : Connexion au serveur MySQL

if(isset($_GET['id_article']))
{

$requete=// À compléter :  requête SQL qui permet de supprimer un article dont l’ id_article  est celui transmis par la page ";


 $conn->query($requete);

header('Location: page_accueil_vend.php');

}
else
{
echo 'Aucune suppression à effectuer';
}
$conn->close();
?>
</body>
</html>
```

Description du projet

Problématique

Vous vous placerez en tant que concepteur indépendant vis-à-vis d'un client qui n'est pas un professionnel du Web mais un commerçant. Il ne peut donc exprimer ses besoins dans un langage technique propre à Internet et à la programmation PHP, dont il ignore tout.

- **Votre travail**

Dans un **premier temps**, vous devez vous mettre à la place d'un client et définir et détailler un projet sur la vente en ligne (vente de produits alimentaires, bricolages, vêtements, etc.).

Dans un **second temps**, vous devez développer une application web pour la gestion des ventes en ligne de votre projet. Comme vous n'allez pas réinventer le concept de commerce en ligne et partir de zéro, vous devez vous inspirerer de ce qui existe déjà. Vous consulterez pour cela des sites de vente en ligne tels que **amazon.fr** pour en étudier le fonctionnement. Il vous faudra en dégager les points essentiels et n'en retenir que ce qui correspond au cas relativement simple de votre client. En particulier, le nombre d'articles vendus par cette PME est relativement limité comparativement à ceux du site d'Amazon. Cela peut entraîner des simplifications dans la structure de la base, comme le fait de ne pas créer de table spéciale pour enregistrer le nom des marques des produits en magasin.

- **Exemple des besoins d'un client (vente de produits électroniques)**

Le client dirige une PME qui vend des produits informatiques et vidéo. Vous devez lui créer un site de commerce en ligne pour vendre ses produits en ligne. Son budget ne lui permettant pas de vous rémunérer en permanence pour maintenir le site, vous lui créez une interface Internet à accès privé lui permettant d'ajouter ou d'enlever des produits dans la base de données sans avoir à passer par la programmation.

Pour accéder à MySQL, utilisez de préférence l'extension mysqli

- **Fonctionnement du site (cas de vente de produits électroniques)**

Le site répond aux conditions suivantes :

1) Dans la page d'accueil, le client recherche un type de produit dans une des catégories informatique, vidéo et divers (pour les accessoires et consommables).
2) Un tri par marque et par prix est proposé pour l'affichage des résultats.
3) Les résultats de la recherche sont affichés en respectant le critère de tri. Chaque produit est suivi d'un lien permettant sa sélection.
4) La sélection d'un produit entraîne sa mise en panier.

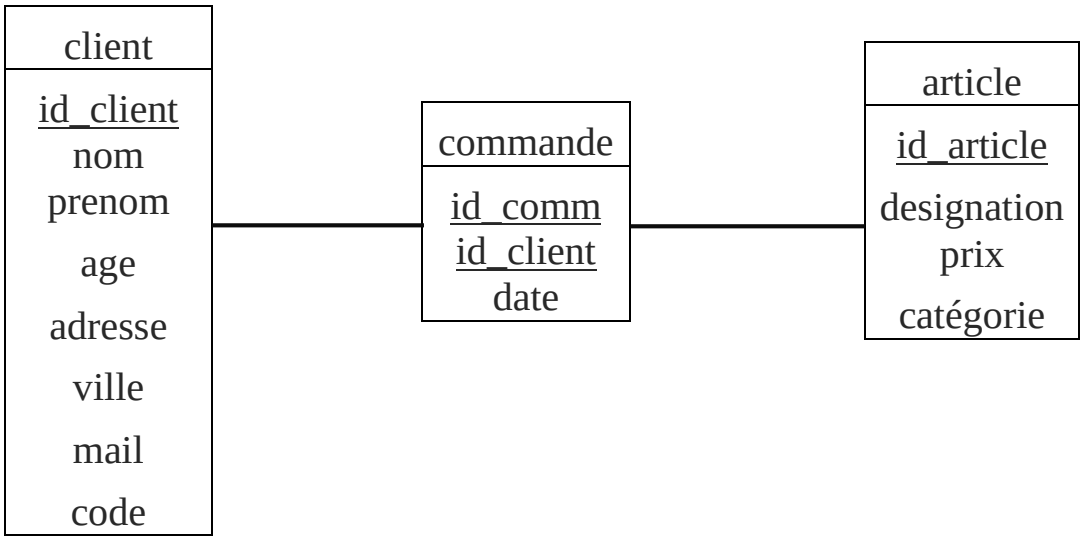
5) Après chaque sélection, le client peut soit rechercher un autre produit, soit terminer sa commande.
6) Dans ce dernier cas, l'ensemble de sa commande est affichée, et vous lui demandez ses coordonnées.
7) Si le client n'est pas encore enregistré, il saisit ses coordonnées complètes. Son adresse e-mail et un mot de passe lui serviront à s'identifier par la suite. Il saisit également l'adresse de livraison, (son adresse). Ces informations sont stockées à l'aide de sessions pour être transmises aux pages suivantes de la procédure d'achat.
8) Si le visiteur est déjà client, il ne saisit que son e-mail et son mot de passe. L'authenticité de ces informations est vérifiée dans la base, et les coordonnées complètes du client sont récupérées. Ces coordonnées sont utilisées pour remplir automatiquement un formulaire identique à celui du visiteur non enregistré. Ces informations sont également stockées à l'aide de sessions.
9) La phase de paiement par carte bancaire n'étant pas réalisable sans une convention bancaire, vous vous contentez de demander la saisie d'un numéro de carte et d'utiliser un algorithme de vérification du numéro.
10) Si le paiement est bien réalisé, vous finalisez la transaction en enregistrant l'ensemble des informations dans les tables client, commande puis confirmer la commande au client.

La base de données MySQL

La première étape dans la création du site web de vente ligne est la modéliser la base de données. Ce site doit interagir avec trois base de données : une base de données ***client***, une base de données ***articles*** et une base de données ***commande***.

1) La base de données ***client*** contient les coordonnées complètes du client.
2) La base de données ***commande*** contient les commandes passées par un client.
3) La base de données ***articles*** contient des informations sur les article mis en vente.

Le Modèle Logique des Données est représenté ci-dessous



client(id_client, nom, prenom, age, adresse, ville, mail, code)

1) La colonne id_client : type MEDIUMINT, PRIMARY KEY, NOT NULL, AUTO_INCREMENT.
2) Les colonnes nom, prenom, adresse, ville, mail : type CHAR, NOT NULL.
3) La colonne age est un entier positif de petite taille, type TINYINT UNSIGNED.

commande(id_comm, id_client, date)

1) La colonne id_comm : type MEDIUMINT, UNSIGNED, NOT NULL et AUTO_INCREMENT.
2) La colonne id_client est une clé étrangère issue de la table client.
3) La clé primaire de la table est composée des champs id_comm et id_client.
4) La colonne date : type DATE, NOT NULL,

article(id_article, designation, prix, catégorie)

5) La colonne id_article: type CHAR(10), NOT NULL et PRIMARY KEY.
6) La colonne designation : type VARCHAR(100), NOT NULL.
7) La colonne prix : type DECIMAL(8,2), NOT NULL.
8) La colonne catégorie : type ENUM. Elle comporte une liste de six valeurs possibles ('Tous', 'Vidéo', 'Photo', 'Téléphonie', 'Informatique', 'Divers'), NOT NULL et la valeur par défaut 'tous'.

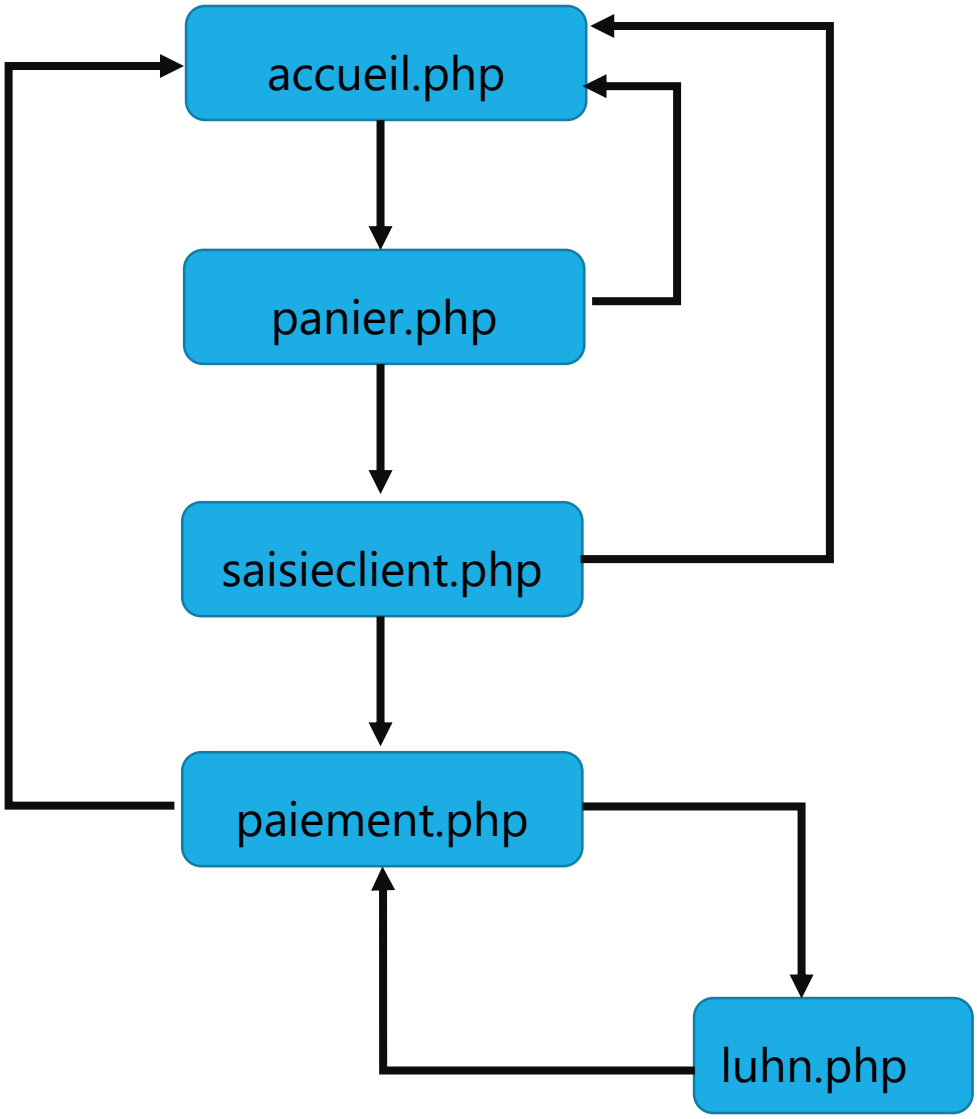
Eléments de la solution proposée

La solution proposée doit comporter :

- Une page d'accueil (***accueil.php***)
- Un fichier ***panier.php*** qui gère les articles mis en panier et qui affiche le contenu du panier.
- Un fichier ***saisieclient.php*** qui permet d'enregistrer les coordonnées d'un client et de les stocker pour des commandes ultérieures.
- Un fichier ***luhn.php*** qui contient la fonction de vérification de la carte bancaire.
- Un fichier ***paiement.php*** qui permet la saisie et la vérification du numéro de carte bancaire puis l'enregistrement des données dans la base MySQL et confirmation de la commande (éventuellement l'envoi d'un mail de confirmation).

Les pages ***acceuil.php***, ***panier.php***, ***saisieclient.php*** et ***paiement.php*** doivent inclure un en-tête et un pied de page.

Le schéma ci-dessous résume les liens entre les pages à créer :



Avant de commencer le projet, il faut organier l'espace de travail. Pour cela il faut :

1. Créer un répertoire principal nommé « SAE23 » dans C:/Xampp/htdocs.

2. Créer trois sous-répertoires nommés « **pages** », « **css** » et « **tables** ».

3. Dans le sous-répertoire « pages », créer un sous-répertoires « images ».

4. Les pages ***entent.html***, ***pied.html***, ***accueil.php***, ***panier.php***, ***saisieclient.php, paiement.php*** et ***luhn.php*** devront se trouver dans le répertoire « **pages**».

5. La feuille de styles nommée « ***style.css*** » doit se trouver dans le sous-répertoire « css ».

6. Les scripts de création des tables mysql doivent se trouver dans le sous-répertoire

« **tables** »

7. Les images devront se trouver dans le sous-répertoire « **images** ».

8. Pour la création des tables il faut exécuter, dans l'ordre, les fichiers suivant

- ***Creat_Article_Table.php***, ***Creat_Client_Table.php*** et ***Creat_Commande_Table. php***.
- Vérifiez la création des trois bases de données en se connectant à **phpMyAdmin** (regardez le contenu de la base **mydb**).

Contenu des pages

entete.html

Le fichier entet.html constitue l'en-tête commun à toutes les pages.

Ci-dessous un exemple d'en-tête. À vous de le personnaliser en ajoutant des images ou vidéo ainsi que du texte et du style.

```
<!DOCTYPE html>
<html>
<head>
<title> Magasin de vente en ligne </title>
<link rel="stylesheet" type="text/css" …… />
</head>
<body>
<div class="bord">
<img src="image.jpeg" alt="Vente en ligne"   />
</div>
</body>
</html>
```

Pour l'inclure dans les pages .php, vous pouvez utiliser l'instruction **include_once** ('entete.html'); au début de chaque page.

pied.html

Le fichier pied.html constitue le pied de page commun à toutes les pages.

Ce fichier doit inclure un bouton qui permet un revoie vers la page d'accueil (***accueil.php***) pour continuer les achats, comme illustré ci-dessous.

Continuer la commande : Page d accueil

Voici le script html à compléter pour avoir le résultat ci-dessus.

```
[…]
<body>
Continuer la commande : <a href=...  à compléter … >
<button ...  à compléter … > ...  à compléter … </button></a>
</body>
[…]
```

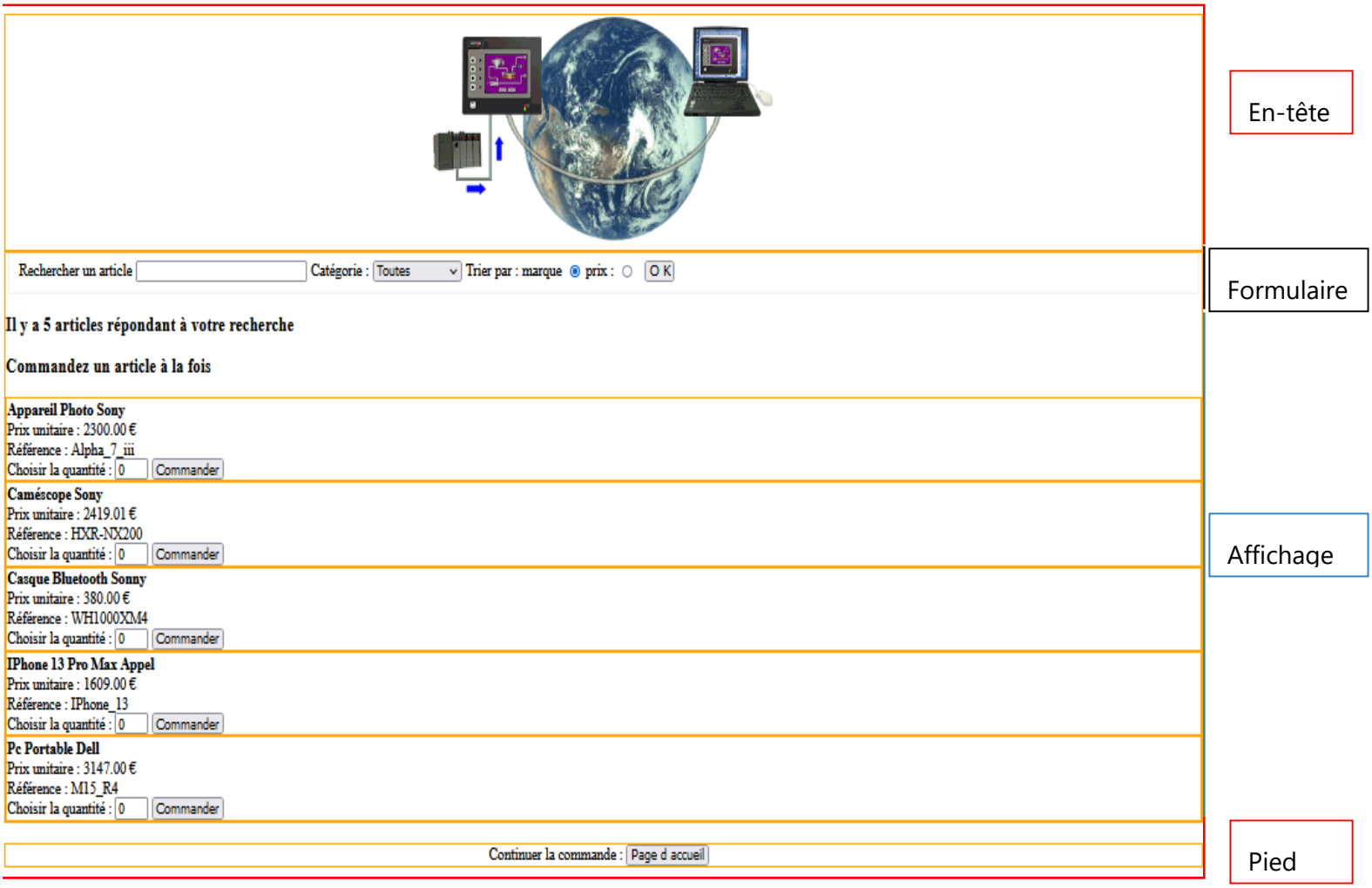
Pour l'inclure dans les pages .php, vous pouvez utiliser l'instruction **include_once**('pied.html'); à la fin de chaque page

accueil.php

Le fichier ***accueil.php*** représente la page d'accueil du site dans laquelle s'effectuent les recherches d'articles. Ce fichier est organisé en quatre blocs :

1) Un bloc **en-tête**
2) Un bloc **Formulaire** qui permet de saisire les choix de l'utilisateur (client)
3) Un bolc **Affichage** qui permet d'afficher les choix de l'utilisateur (client)
4) Un boloc **pied de page**

Ces quatre blocs sont illustrés ci-dessous :



accueil.php

Le bloc formulaire doit contenir :

1) Un espace de saisie pour la rechercher d'un article par mot-clé
   Utilisez la balise <input type= ....>
2) Une espace pour la recherche par catégorie qui inclue les options suivantes : Toutes, Vidéos, Photo, Informatique et Divers.
   Utilisez les balises <select type=.... > et <option value=....>
3) Un espace pour affiner la recherche par un tri soit par marque soit par prix
   Utilisez la balise <input type=....>
4) Un bouton pour la validation du formulaire
   Utilisez la balise <input type=....>

Le bolc Affichage permet d'afficher les choix de l'utilisateur suite un clic sur OK. Ce bloc doit inclure :

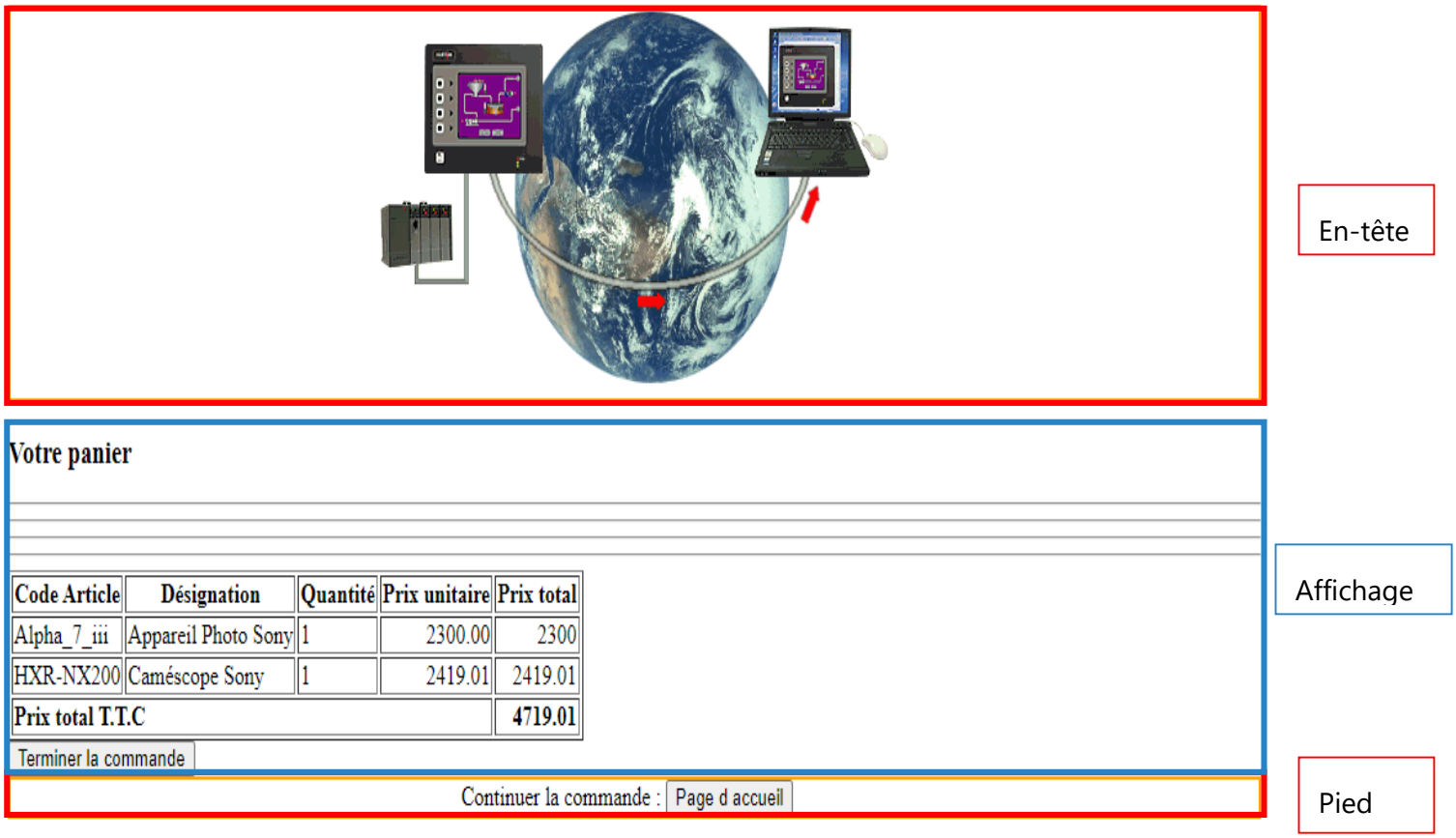
1) Un partie qui permet la récupération des saisies faites dans le formulaire.
2) Une partie pour la création de la requète Mysql.
3) Une partie pour l'afficahge des résultats de la requète. Cette partie doit contenir un chant de saisie de la quantitie d'acticle à comander ainsi qu'un bouton pour la commande qui revoie vers la page panier.php

panier.php

Le fichier ***panier.php*** gère les articles mis en panier et affiche le contenu du panier. Ce fichier est organisé en trois blocs :

1) Un bloc en-tete
2) Un bolc Affichage qui affiche les articles mis en panier.
3) Un boloc pied de page

Ces quatre blocs sont illustrés ci-dessous



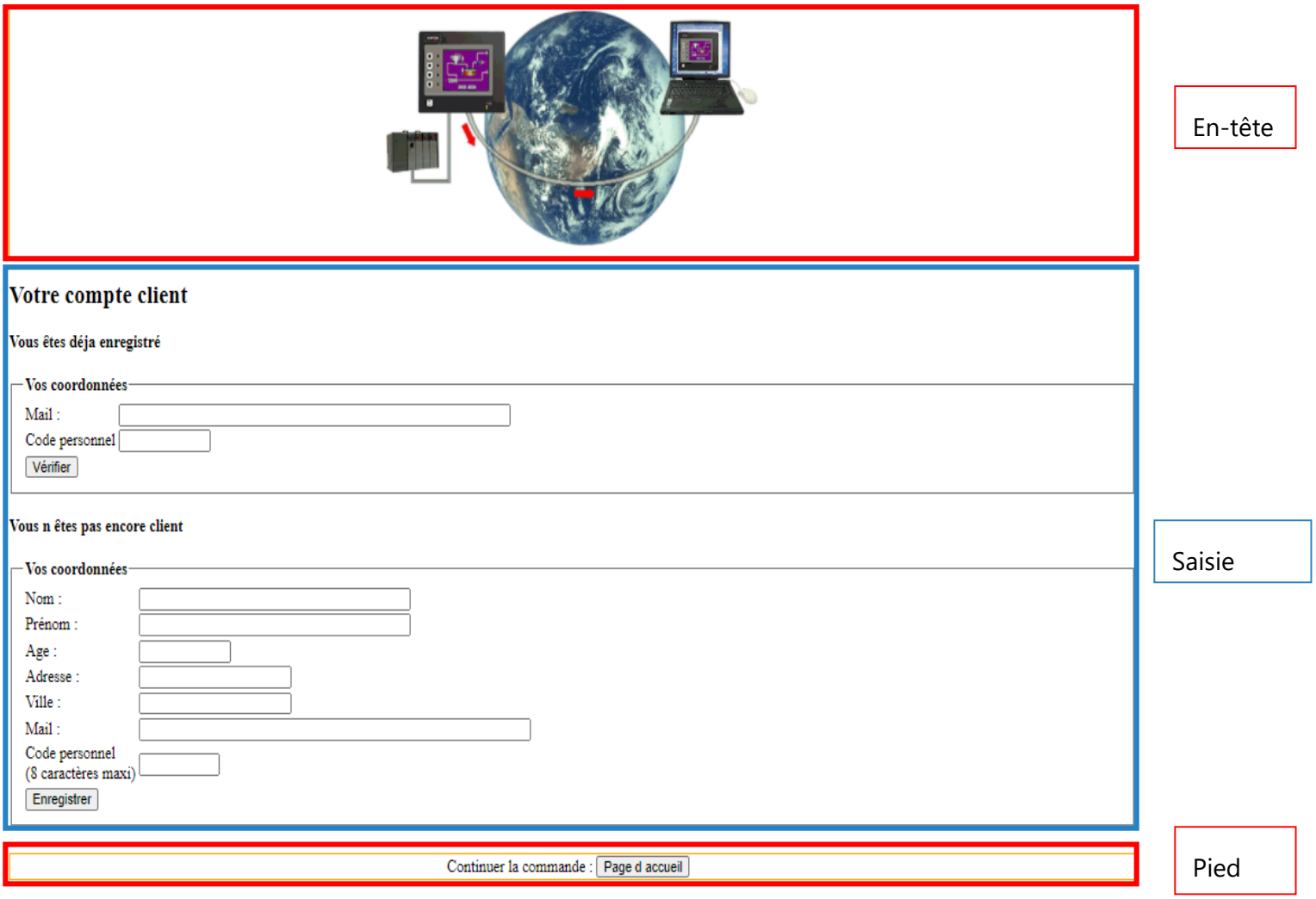
Le bolc ***Affichage*** permet d'afficher les articles mis en panier sous forme d'un tableau conteant le l'Id_article, la désignation (marque) la quantitie le prix total par produit et prix total du pannier en TTC, comme ilustré sur la figure ci-dessus. Il faut prévoir un bouton (terminer la commande) qui permet un revoi vers le fichier ***saisieclient.php***.

saisieclient.php

Le fichier ***saisieclient.php*** permet de passer la commande si on est déjà client ou d'enregistrer les coordonnées d'un client et de les stocker pour des commandes ultérieures. Ce fichier est organisé en trois blocs :

1) Un bloc en-tete
2) Un bolc Saisie
3) Un boloc pied de page

Ces quatre blocs sont illustrés ci-dessous



saisieclient.php

Le bolc saisie comprend deux formulaires :

1) Un formulaire de saisie pour les utilisateurs qui sont déjà client. Ce formulaire doit un inclure deux chants de saisies (un pour saisir l'adresse mail, l'autre pour saisir le code confidentiel), et un bouton qui renvoie vers la page ***paiement.php*** si les informations saisies sont correctes sinon une alerte est générée pour inviter l'utilisateur à ressaisir ses informations ou créer un compte client.
2) Un formulaire pour la création d'un compte client. Ce formulaire doit inclure cinq chants de saisies (comme illustré sur la figure ci-dessus) ainsi qu'un bouton qui permet un renvoie vers la page ***paiement.php***.

paiement.php

Le fichier ***paimenet.php*** permet la saisie et la vérification du numéro de la carte bancaire. Si le numéro de carte bancaire saisie est valide, Les données nouveau client sont insérées dans la base de données client et un message de confirmation de la commande est affiché, sinon une alerte est générée pour informer l'utilisateur que sa carte bancaire est invalide.

Ce fichier est organisé en trois blocs :

1) Un bloc en-tete
2) Un bolc de saisie du numéro de la carte bacaire.
3) Un boloc pied de page

Ces quatre blocs sont illustrés ci-dessous

Le bloc de saisie est composé de deux partie :

1) Une partie formulaire pour la saisie du numéro de la carte bancaire à 16 chiffre et un bouton pour la validation du formulaire
2) Une partie de vérification du numéro de la carte bancaire. Si le numéro de carte bancaire saisi est valide, Les données nouveau client sont insérées dans la base de données client et un message de confirmation de la commande est affiché, sinon une alerte est générée pour informer l'utilisateur que sa carte bancaire est invalide.

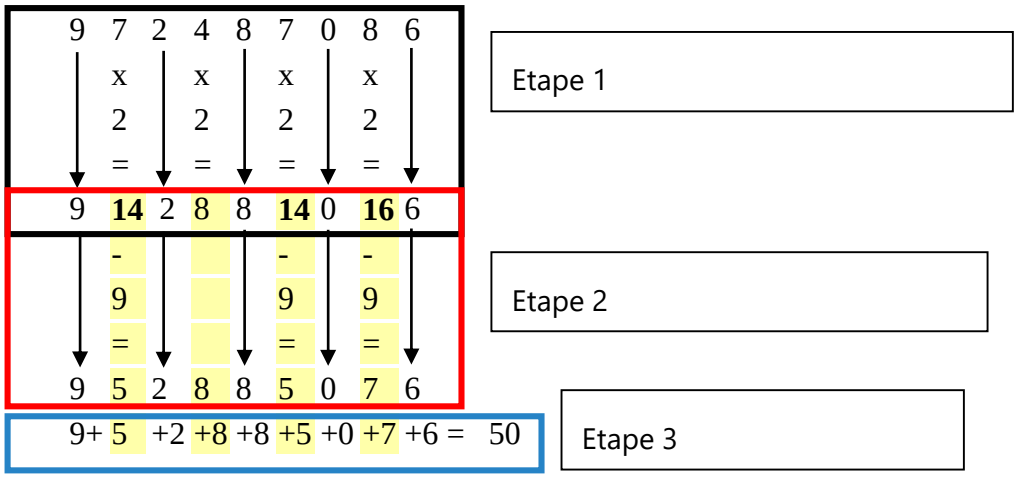
Luhn.php

Le script ***luhn.php*** contient la fonction de vérification de la carte bancaire. Cette fonction est basée sur la formule de Luhn qui est utilisée dans applications de cryptologie. L'algorithme de Luhn ou la formule de Luhn, aussi connu comme l'algorithme « modulo 10 » ou « mod 10 », fut développé dans les années 1960 comme une méthode de validation d'identification de nombres. C'est une simple formule de somme de contrôle (Checksum) utilisée pour valider une variété de numéros de comptes, comme les numéros de cartes bancaires, les numéros d'assurance sociale canadiens ainsi que pour le calcul de validité d'un numéro SIRET.

L'algorithme procède en trois étapes.

1. L'algorithme multiplie par deux un chiffre sur deux, en commençant par l'avant dernier et en se déplaçant de droite à gauche. Si le double d'un chiffre est plus grand que neuf (par exemple 2 × 8 = 16), alors il faut le ramener à un chiffre entre 1 et 9 en prenant son reste dans la division euclidienne par 9. Pour cela, il y a deux manières de le faire (pour un résultat identique) :
    1. Soit on additionne les chiffres composant le double. Dans l'exemple du chiffre 8, 2 × 8 = 16, puis on additionne les chiffres 1 + 6 = 7.
    2. Soit on soustrait 9 au double. Avec le même exemple, 16 − 9 = 7.
2. La somme de tous les chiffres obtenus est effectuée.
3. Le résultat est divisé par 10. Si le reste de la division est égal à zéro, alors le nombre original est valide.

Exemple : soit le numéro 972487086



Le numéro 972487086 est valide car 50 est divisible sur 10

Le script ***luhn.php*** est constituer de la fonction de vérification qui retourne ***true*** si le numéro de la carte bancaire est valide ou ***false*** sinon.